# CS 229, Winter 2026
# Problem Set #2

---

**Due Wednesday, February 4 at 11:59 pm on Gradescope.**

**Notes:** (1) These questions require thought, but do not require long answers. Please be as concise as possible.

(2) If you have a question about this homework, we encourage you to post your question on our Ed forum, at `https://edstem.org/us/courses/90830/discussion/`.

(3) If you missed the first lecture or are unfamiliar with the collaboration or honor code policy, please read the policy on the course website before starting work.

(4) For the coding problems, you may not use any libraries except those defined in the provided `environment.yml` file. In particular, ML-specific libraries such as scikit-learn are not permitted.

(5) The due date is Wednesday, February 4 at 11:59 pm. If you submit after Wednesday, February 4 at 11:59 pm, you will begin consuming your late days. The late day policy can be found in the course website: **Course Logistics and FAQ**.

All students must submit an electronic PDF version of the written question including plots generated from the codes. We highly recommend typesetting your solutions via LaTeX. All students must also submit a zip file of their source code to Gradescope, which should be created using the `make_zip.py` script. You should make sure to (1) restrict yourself to only using libraries included in the `environment.yml` file, and (2) make sure your code runs without errors. Your submission may be evaluated by the auto-grader using a private test set, or used for verifying the outputs reported in the writeup. Please make sure that your PDF file and zip file are submitted to the corresponding Gradescope assignments respectively. We reserve the right to not give any points to the written solutions if the associated code is not submitted.

**Honor code:** We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solution independently, and without referring to written notes from the joint session. Each student must understand the solution well enough in order to reconstruct it by him/herself. It is an honor code violation to copy, refer to, or look at written or code solutions from a previous year, including but not limited to: official solutions from a previous year, solutions posted online, and solutions you or someone else may have written up in a previous year. Furthermore, it is an honor code violation to post your assignment solutions online, such as on a public git repo. We run plagiarism-detection software on your code against past solutions as well as student submissions from previous years. Please take the time to familiarize yourself with the Stanford Honor Code[1] and the Stanford Honor Code[2] as it pertains to CS courses.

[1] https://communitystandards.stanford.edu/policies-and-guidance/honor-code
[2] https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1164/handouts/honor-code.pdf

1. **[10 points] A Bayesian Interpretation of Lasso**

Let's start with the probabilistic interpretation of least squares. We have data $x \in \mathbb{R}^d$ and labels $y \in \mathbb{R}$. We make the assumption that our observed labels are noised by an additive term $z \sim \mathcal{N}(0, \sigma^2)$, so $y = (w^*)^\top x + z$. We then have

$$P(y \mid x;\, w^*, \sigma^2) \sim \mathcal{N}((w^*)^\top x, \sigma^2)$$

for the true weight vector $w^*$. Note that, when collecting our dataset, we assume that the labels are drawn i.i.d.

Recall that maximum likelihood estimation finds the parameters $\hat{w}_{\text{MLE}}$ that maximize the likelihood of the data $\{(x^{(i)}, y^{(i)})_{i=1}^n\} \subset \mathbb{R}^d \times \mathbb{R}$ in the probabilistic model that we have set. Concretely, this is given by

$$\begin{aligned}
\hat{w}_{\text{MLE}} &= \operatorname*{argmax}_w \mathcal{L}(w) \\
&= \operatorname*{argmax}_w P(y^{(1)}, \ldots, y^{(n)} \mid x^{(1)}, \ldots, x^{(n)}; w, \sigma^2) \\
&= \operatorname*{argmax}_w \prod_{i=1}^n P(y^{(i)} \mid x^{(i)}; w, \sigma^2).
\end{aligned}$$

Maximum likelihood estimates (MLE) can overfit by picking parameters that mirror the training data. To prevent this, we take an *independent* Laplace prior on $w_j \sim \text{Laplace}(0, t)$ to try to shrink the parameters closer to zero and encourage sparsity (exactly zero weights). The following is the density function of the Laplace distribution.

$$\begin{aligned}
P(w_j) &= \frac{1}{2t} \exp\left(-\frac{|w_j|}{t}\right) \\
P(w) &= \prod_{j=1}^d P(w_j) = \left(\frac{1}{2t}\right)^d \cdot \exp\left(-\frac{1}{t}\sum_{j=1}^d |w_j|\right).
\end{aligned}$$

The parameter $t$ is a scale parameter that describes spread of the Laplace distribution. We will see that imposing this prior results in an optimization problem that is equivalent to minimizing the Lasso cost function when we perform maximum a posteriori estimation.

When working in a Bayesian framework, we instead focus on the posterior distribution of the parameters conditioned on the data, $P(w \mid y^{(1)}, \ldots, y^{(n)}, x^{(1)}, \ldots, x^{(n)}, \sigma^2)$. To pick a single model, we can choose the $w$ that is most likely according to the posterior,

$$\hat{w}_{\text{MAP}} = \operatorname*{argmax}_w P(w \mid y^{(1)}, \ldots, y^{(n)}, x^{(1)}, \ldots, x^{(n)}, \sigma^2)$$

We call $\hat{w}_{\text{MAP}}$ the maximum a posteriori (MAP) estimate.

(a) [5 points] Write the log-posterior-likelihood, $l(w) = \log P(w \mid y^{(1)}, \ldots, y^{(n)}, x^{(1)}, \ldots, x^{(n)}, \sigma^2)$, for this MAP estimate up to an additive constant that does not depend on $w$.

**Hint:** Use Bayes' rule, and think about the denominator. In addition, you should treat the data points $x^{(i)}$ as fixed (constant).

(b) [5 points] Now show that the MAP objective is equivalent to minimizing the following LASSO cost. Additionally, identify the constant $\lambda$ in terms of parameters present in the MAP objective. Note that $\|w\|_1 = \sum_{j=1}^{d} |w_j|$.

$$J(w) = \sum_{i=1}^{n} \left(y^{(i)} - w^\top x^{(i)}\right)^2 + \lambda \|w\|_1.$$

2. **[18 points] Bias and Variance of the Ridge and OLS Estimators**

We will take a similar setup as in the previous problem. Given training data $\{(x^{(i)}, y^{(i)})\}_{i=1}^n \subset \mathbb{R}^d \times \mathbb{R}$, we write the statistical model in matrix-vector form as

$$Y = Xw^* + z,$$

where $w^* \in \mathbb{R}^d$ is the true parameter we are trying to estimate, $X \in \mathbb{R}^{n \times d}$ is the fixed design matrix, $\mathbb{E}[z] = 0$, $\mathrm{Cov}(z) = \sigma^2 I_n$, and $Y \in \mathbb{R}^n$ is the random variable representing our labels. Note that we have eased the assumptions on $z$ to not necessarily be Gaussian or independent.

Throughout this question, you may assume that $X$ is full column rank. Given a realization of the labels $Y = y$, recall these two estimators:

$$\hat{w}_{\text{ols}} = \arg\min_{w \in \mathbb{R}^d} \|Xw - y\|_2^2,$$
$$\hat{w}_{\text{ridge}} = \arg\min_{w \in \mathbb{R}^d} \|Xw - y\|_2^2 + \lambda \|w\|_2^2.$$

Also recall that the solutions are

$$\hat{w}_{\text{ols}} = \left(X^\top X\right)^{-1} X^\top y,$$
$$\hat{w}_{\text{ridge}} = \left(X^\top X + \lambda I_d\right)^{-1} X^\top y.$$

(a) **[5 points]** Let $\hat{w} \in \mathbb{R}^d$ denote any estimator of $w^*$. Define the MSE (mean squared error) of $\hat{w}$ as

$$\mathrm{MSE}(\hat{w}) := \mathbb{E}\left\|\hat{w} - w^*\right\|_2^2.$$

Define $\hat{\mu} := \mathbb{E}[\hat{w}]$. Show that the MSE decomposes as such:

$$\mathrm{MSE}(\hat{w}) = \underbrace{\|\hat{\mu} - w^*\|_2^2}_{\mathrm{Bias}(\hat{w})^2} + \underbrace{\mathrm{tr}(\mathrm{Cov}(\hat{w}))}_{\mathrm{Var}(\hat{w})}.$$

Note that this is a multivariate generalization of the bias-variance decomposition we have seen previously.

*Hint:* The inner product of two vectors is the trace of their outer product. Also, expectation and trace commute, so $\mathbb{E}[\mathrm{tr}(A)] = \mathrm{tr}(\mathbb{E}[A])$ for any square matrix $A$.

(b) **[3 points]** Show that

$$\mathbb{E}[\hat{w}_{\text{ols}}] = w^*,$$
$$\mathbb{E}[\hat{w}_{\text{ridge}}] = (X^\top X + \lambda I_d)^{-1} X^\top X w^*.$$

That is, $\mathrm{Bias}(\hat{w}_{\text{ols}}) = 0$, and hence $\hat{w}_{\text{ols}}$ is an *unbiased* estimator of $w^*$, whereas $\hat{w}_{\text{ridge}}$ is a *biased* estimator of $w^*$.

(c) **[8 points]** Let $\{\gamma_i\}_{i=1}^d$ denote the $d$ eigenvalues of the matrix $X^\top X$. Define the *total variance* of a vector $\hat{w}$ as

$$\mathrm{Var}(\hat{w}) = \mathrm{tr}(\mathrm{Cov}(\hat{w})).$$

Show that

$$\mathrm{Var}(\hat{w}_{\text{ols}}) = \sigma^2 \sum_{i=1}^d \frac{1}{\gamma_i}, \qquad \mathrm{Var}(\hat{w}_{\text{ridge}}) = \sigma^2 \sum_{i=1}^d \frac{\gamma_i}{(\gamma_i + \lambda)^2}.$$

Finally, use these formulas to conclude that

$$\text{Var}(\hat{w}_{\text{ridge}}) < \text{Var}(\hat{w}_{\text{ols}}).$$

Note that this is opposite of the relationship between the bias terms.

*Hint:* Remember the relationship between the trace and the eigenvalues of a matrix. Also, for the ridge variance, consider writing $X^\top X$ in terms of its eigendecomposition $X^\top X = U\Gamma U^\top$. Note that $X^\top X + \lambda I_d$ has the eigendecomposition $U(\Gamma + \lambda I_d)U^\top$.

(d) [2 points] Use the previous parts to briefly explain why the ridge estimator may result in a lower MSE than OLS despite being biased.

3. **[12 points] Logistic Regression: Training stability**

In this problem, we will be delving deeper into the workings of logistic regression. The goal of this problem is to help you develop your skills debugging machine learning algorithms (which can be very different from debugging software in general).

We have provided an implementation of logistic regression in `src/stability/stability.py`, and two labeled datasets $A$ and $B$ in `src/stability/ds1_a.csv` and `src/stability/ds1_b.csv`.

Please do not modify the code for the logistic regression training algorithm for this problem. First, run the given logistic regression code to train two different models on $A$ and $B$. You can run the code by simply executing `python stability.py` in the `src/stability` directory.

(a) [2 points] What is the most notable difference in training the logistic regression model on datasets $A$ and $B$?

(b) [5 points] Investigate why the training procedure behaves unexpectedly on dataset $B$, but not on $A$. Provide hard evidence (in the form of math, code, plots, etc.) to corroborate your hypothesis for the misbehavior. Remember, you should address why your explanation does *not* apply to $A$.

**Hint**: The issue is not a numerical rounding or over/underflow error. Plotting the data should prove fruitful.

(c) [5 points] For each of these possible modifications, state whether or not it would lead to the provided training algorithm converging on datasets such as $B$. Justify your answers.

i. Using a different constant learning rate.

ii. Decreasing the learning rate over time (e.g. scaling the initial learning rate by $1/t^2$, where $t$ is the number of gradient descent iterations thus far).

iii. Linear scaling of the input features.

iv. Adding a regularization term $\|\theta\|_2^2$ to the loss function.

v. Adding zero-mean Gaussian noise to the training data or labels.

4. **[12 points] Batched Neural Network Gradients**

**Note:** This question will be easier to work through once backpropagation is covered in lecture. Your derivations in this question should prove helpful for Q5.

In this question you will derive batched gradients for common neural network components. Your answers must use mini-batch matrices (no summations over the batch dimension), and should be expressed using matrix operations and elementwise products.

**Hint:** recall the multivariate chain rule. Let $f : \mathbb{R}^{n\times k} \to \mathbb{R}$, $g : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times k}$, $A \in \mathbb{R}^{n\times d}$, and $B = g(A)$. Then the gradient $\frac{\partial f(g(A))}{\partial A}$ is given by

$$\left[\frac{\partial f(g(A))}{\partial A}\right]_{ij} = \frac{\partial f(g(A))}{\partial A_{ij}} = \sum_{k,l} \frac{\partial f(g(A))}{\partial B_{kl}} \frac{\partial B_{kl}}{\partial A_{ij}}.$$

**Hint:** gradients should have the same shape as the input.

(a) [3 points] Let $Z \in \mathbb{R}^{n\times k}$ be a batched pre-activation matrix and let $A$ be the elementwise application of the sigmoid function to $Z$, i.e. $A_{ij} = \sigma_{\text{sigmoid}}(Z)_{ij} = h(Z_{ij})$, where $h$ is the sigmoid function in $\mathbb{R} \to \mathbb{R}$. Suppose the upstream gradient $\frac{\partial L}{\partial A}$ is given and has the same shape as $A$. Derive a batched expression for $\frac{\partial L}{\partial Z}$ in terms of $\frac{\partial L}{\partial A}$ and $Z$. Your answer must use elementwise operations and should not iterate over training examples.

**Hint:** you may use elementwise multiplication (Hadamard product).

(b) [7 points] Consider a fully-connected (affine) layer with batched input $X \in \mathbb{R}^{n\times d}$, weights $W \in \mathbb{R}^{d\times k}$, and bias row vector $b \in \mathbb{R}^{1\times k}$. The pre-activation is

$$Z = XW + \mathbf{1}b,$$

where $\mathbf{1} \in \mathbb{R}^{n\times 1}$ is a column of ones (so $\mathbf{1}b$ broadcasts $b$ across all $n$ examples in the batch). Let the upstream $\frac{\partial L}{\partial Z} \in \mathbb{R}^{n\times k}$ be given. Derive batched expressions for

$$\frac{\partial L}{\partial X}, \quad \frac{\partial L}{\partial W}, \quad \frac{\partial L}{\partial b}.$$

Your answers must be expressed using matrix operations and should not iterate over training examples.

**A word of caution:** A *very* common pitfall occurs when students try to explicitly compute $\frac{\partial Z}{\partial X}$, or any of the other intermediate derivatives. In the case of $\frac{\partial Z}{\partial X}$, this derivative is a linear operator in the space of $\mathbb{R}^{n\times d} \to \mathbb{R}^{n\times k}$ functions, meaning that the derivative itself is $(n \times k \times n \times d)$-dimensional. Rather than trying to construct such an object, try using the expression of multivariate chain rule provided above.

(c) [2 points] Suppose that there are layers before the current one, meaning that $X$ is the output of the previous layer. In part (b), you computed $\frac{\partial L}{\partial X}$ for the current layer. However, $X$ is the input matrix, not a set of parameters that we are optimizing. So we will not use $\frac{\partial L}{\partial X}$ to update the parameters of the current layer. Briefly explain why we care about $\frac{\partial L}{\partial X}$ when there are layers before the current one.

5. **[30 points] Neural Networks: MNIST image classification**

**Note:** This question will be easier to work through once backpropagation is covered in lecture.

In this problem, you will implement a simple neural network to classify grayscale images of handwritten digits (0 - 9) from the MNIST dataset. The dataset contains 60,000 training images and 10,000 testing images of handwritten digits, 0 - 9. Each image is 28×28 pixels in size, and is generally represented as a flat vector of 784 numbers. It also includes labels for each example, a number indicating the actual digit (0 - 9) handwritten in that image. A sample of a few such images are shown below.

The data and starter code for this problem can be found in

- `src/mnist/nn.py`
- `src/mnist/images_train.csv`
- `src/mnist/labels_train.csv`
- `src/mnist/images_test.csv`
- `src/mnist/labels_test.csv`

The starter code splits the set of 60,000 training images and labels into a set of 50,000 examples as the training set, and 10,000 examples for dev set.

To start, you will implement a neural network with a single hidden layer and cross entropy loss, and train it with the provided data set. You will use the sigmoid function as activation for the hidden layer and use the cross-entropy loss for multi-class classification. Recall that for a single example $(x, y)$, the cross entropy loss is:

$$\ell_{\text{CE}}(\bar{h}_\theta(x), y) = -\log\left(\frac{\exp(\bar{h}_\theta(x)_y)}{\sum_{s=1}^{k}\exp(\bar{h}_\theta(x)_s)}\right),$$

where $\bar{h}_\theta(x) \in \mathbb{R}^k$ is the logits, i.e., the output of the the model on a training example $x$, $\bar{h}_\theta(x)_y$ is the $y$-th coordinate of the vector $\bar{h}_\theta(x)$ (recall that $y \in \{1, \ldots, k\}$ and thus can serve as an index.)

For clarity, we provide the forward propagation equations below for the neural network with a single hidden layer. We have labeled data $(x^{(i)}, y^{(i)})_{i=1}^n$, where $x^{(i)} \in \mathbb{R}^d$, and $y^{(i)} \in \{1, \ldots, k\}$ is ground truth label. Let $m$ be the number of hidden units in the neural network, so that weight matrices $W^{[1]} \in \mathbb{R}^{d \times m}$ and $W^{[2]} \in \mathbb{R}^{m \times k}$.[3] We also have biases $b^{[1]} \in \mathbb{R}^m$ and $b^{[2]} \in \mathbb{R}^k$. The parameters of the model $\theta$ is $(W^{[1]}, W^{[2]}, b^{[1]}, b^{[2]})$. The forward propagation equations for a single input $x^{(i)}$ then are:

$$\begin{aligned} a^{(i)} &= \sigma\left({W^{[1]}}^\top x^{(i)} + b^{[1]}\right) \in \mathbb{R}^m \\ \bar{h}_\theta(x^{(i)}) &= {W^{[2]}}^\top a^{(i)} + b^{[2]} \in \mathbb{R}^k \\ h_\theta(x^{(i)}) &= \text{softmax}(\bar{h}_\theta(x^{(i)})) \in \mathbb{R}^k \end{aligned}$$

where $\sigma$ is the sigmoid function.

For $n$ training examples, we average the cross entropy loss over the $n$ examples.

$$J(W^{[1]}, W^{[2]}, b^{[1]}, b^{[2]}) = \frac{1}{n}\sum_{i=1}^n \ell_{\text{CE}}(\bar{h}_\theta(x^{(i)}), y^{(i)}) = -\frac{1}{n}\sum_{i=1}^n \log\left(\frac{\exp(\bar{h}_\theta(x^{(i)})_{y^{(i)}})}{\sum_{s=1}^k \exp(\bar{h}_\theta(x^{(i)})_s)}\right).$$

Suppose $e_y \in \mathbb{R}^k$ is the one-hot embedding/representation of the discrete label $y$, where the $y$-th entry is 1 and all other entries are zeros. We can also write the loss function in the following way:

$$J(W^{[1]}, W^{[2]}, b^{[1]}, b^{[2]}) = -\frac{1}{n}\sum_{i=1}^n e_{y^{(i)}}^\top \log\left(h_\theta(x^{(i)})\right).$$

Here $\log(\cdot)$ is applied entry-wise to the vector $h_\theta(x^{(i)})$. The starter code already converts labels into one-hot representations for you.

Instead of batch gradient descent or stochastic gradient descent, the common practice is to use mini-batch gradient descent for deep learning tasks. Concretely, we randomly sample $B$ examples $(x^{(i_k)}, y^{(i_k)})_{k=1}^B$ from $(x^{(i)}, y^{(i)})_{i=1}^n$. In this case, the mini-batch cost function with batch-size $B$ is defined as follows:

$$J_{MB} = \frac{1}{B}\sum_{k=1}^B \ell_{\text{CE}}(\bar{h}_\theta(x^{(i_k)}), y^{(i_k)})$$

where $B$ is the batch size, i.e., the number of training examples in each mini-batch.

(a) **[5 points]**
Let $t \in \mathbb{R}^k, y \in \{1, \ldots, k\}$ and $p = \text{softmax}(t)$. Prove that

$$\frac{\partial \ell_{\text{CE}}(t, y)}{\partial t} = p - e_y \in \mathbb{R}^k, \tag{1}$$

where $e_y \in \mathbb{R}^k$ is the one-hot embedding of $y$, (where the $y$-th entry is 1 and all other entries are zeros.) As a direct consequence,

[3] Please note that the dimension of the weight matrices is different from those in the lecture notes, but we also multiply ${W^{[1]}}^\top$ instead of $W^{[1]}$ in the matrix multiplication layer. Such a change of notation is mostly for some consistence with the convention in the code.

$$\frac{\partial \ell_{\text{CE}}(\bar{h}_\theta(x^{(i)}), y^{(i)})}{\partial \bar{h}_\theta(x^{(i)})} = \text{softmax}(\bar{h}_\theta(x^{(i)})) - e_{y^{(i)}} = h_\theta(x^{(i)}) - e_{y^{(i)}} \in \mathbb{R}^k \tag{2}$$

where $\bar{h}_\theta(x^{(i)}) \in \mathbb{R}^k$ is the input to the softmax function, i.e.

$$h_\theta(x^{(i)}) = \text{softmax}(\bar{h}_\theta(x^{(i)}))$$

(Note: in deep learning, $\bar{h}_\theta(x^{(i)})$ is sometimes referred to as the "logits".)

(b) **[15 points]**

Implement both forward-propagation and back-propagation for the above loss function $J_{MB} = \frac{1}{B}\sum_{k=1}^{B} \ell_{\text{CE}}(\bar{h}_\theta(x^{(i_k)}), y^{(i_k)})$. Initialize the weights of the network by sampling values from a standard normal distribution. Initialize the bias/intercept term to 0. Set the number of hidden units to be 300, and learning rate to be 5. Set $B = 1,000$ (mini batch size). This means that we train with 1,000 examples in each iteration. Therefore, for each epoch, we need 50 iterations to cover the entire training data. The images are pre-shuffled. So you don't need to randomly sample the data, and can just create mini-batches sequentially.

Train the model with mini-batch gradient descent as described above. Run the training for 30 epochs. At the end of each epoch, calculate the value of loss function averaged over the entire training set, and plot it (y-axis) against the number of epochs (x-axis). In the same image, plot the value of the loss function averaged over the dev set, and plot it against the number of epochs.

Similarly, in a new image, plot the accuracy (on y-axis) over the training set, measured as the fraction of correctly classified examples, versus the number of epochs (x-axis). In the same image, also plot the accuracy over the dev set versus number of epochs.

**Submit the two plots (one for loss vs epoch, another for accuracy vs epoch) in your writeup.**

Also, at the end of 30 epochs, save the learnt parameters (i.e., all the weights and biases) into a file, so that next time you can directly initialize the parameters with these values from the file, rather than re-training all over. You do NOT need to submit these parameters.

**Hint:** Be sure to vectorize your code as much as possible! Training can be very slow otherwise. For better vectorization, use one-hot label encodings in the code ($e_y$ in part (a)).

(c) **[7 points]** Now add a regularization term to your cross entropy loss. The loss function will become

$$J_{MB} = \left(\frac{1}{B}\sum_{k=1}^{B} \ell_{\text{CE}}(\bar{h}_\theta(x^{(i_k)}), y^{(i_k)})\right) + \lambda\left(||W^{[1]}||^2 + ||W^{[2]}||^2\right)$$

Be careful not to regularize the bias/intercept term. Set $\lambda$ to be 0.0001. Implement the regularized version and plot the same figures as part (a). Be careful NOT to include the regularization term to measure the loss value for plotting (i.e., regularization should only be used for gradient calculation for the purpose of training).

**Submit the two new plots obtained with regularized training (i.e loss (without regularization term) vs epoch, and accuracy vs epoch) in your writeup.**

**Compare the plots obtained from the regularized model with the plots obtained from the non-regularized model, and summarize your observations in a couple of sentences.**

As in the previous part, save the learnt parameters (weights and biases) into a different file so that we can initialize from them next time.

(d) **[3 points]** All this while you should have stayed away from the test data completely. Now that you have convinced yourself that the model is working as expected (i.e., the observations you made in the previous part matches what you learnt in class about regularization), it is finally time to measure the model performance on the test set. Once we measure the test set performance, we report it (whatever value it may be), and NOT go back and refine the model any further.

Initialize your model from the parameters saved in part (b) (i.e., the non-regularized model), and evaluate the model performance on the test data. Repeat this using the parameters saved in part (c) (i.e., the regularized model).

Report your test accuracy for both regularized model and non-regularized model. Briefly (in one sentence) explain why this outcome makes sense. You should have accuracy close to 0.92870 without regularization, and 0.96760 with regularization. Note: these accuracies assume you implement the code with the matrix dimensions as specified in the comments, which is not the same way as specified in your code. Even if you do not precisely these numbers, you should observe good accuracy and better test accuracy with regularization.